AHMADU BELLO UNIVERSITY, ZARIA

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

COEN838: ADVANCED SOFTWARE ENGINEERING

Project Examination, Question 4

OFFLINE-FIRST CRYPTOGRAPHIC QR IDENTITY WALLET

FOR RURAL PUBLIC SERVICES

PROJECT, RISK AND MAINTENANCE PLAN

Document QRW-PRM-05 · Version 1.0

BY

ALIYU SARKI ILIYASU

P25EGCP8040

November 2026

DOCUMENT CONTROL

Document details

Item	Detail
Document	Project, Risk and Maintenance Plan (QRW-PRM-05)
Project	Offline-First Cryptographic QR Identity Wallet for Rural Public Services
Course	COEN838 Advanced Software Engineering, Ahmadu Bello University, Zaria
Author	Aliyu Sarki Iliyasu (P25EGCP8040)
Version / date	1.0 · November 2026
Source code	https://github.com/sarki94/qr-identity-wallet
Related documents	QRW-SRS-01 Requirements · QRW-SEC-03 Security · QRW-TST-04 Test Plan & Report

Revision history

Version	Date	Change
1.0	4 Nov 2026	First complete project, risk and maintenance plan

TABLE OF CONTENTS

1. INTRODUCTION....1
1.1 Purpose....1
1.2 References....1
2. PROJECT MANAGEMENT....2
2.1 Process Model....2
2.2 Work Breakdown and Estimates....2
2.3 Scope Management....3
2.4 Tools....3
3. DELIVERY METRICS....4
3.1 Earned Value....4
3.2 Burndown....5
3.3 Velocity....5
3.4 Technical Debt Ratio....5
3.5 Effort Estimate with COCOMO....6
3.6 Quality Metrics....7
4. RISK MANAGEMENT....8
4.1 Process....8
4.2 Risk Register....8
4.3 Risk Matrix....10
4.4 Discussion of the Main Risks....10
5. MAINTENANCE PLAN....11
5.1 Maintenance Model....11
5.2 Anticipated Changes....11
5.3 Maintenance Process....12
5.4 Schedule and Effort Allocation....12
6. SUMMARY....13

1. INTRODUCTION

1.1 Purpose

This document covers the management side of the project. It describes the development process and how the work was planned and tracked; reports the delivery metrics (earned value, burndown, velocity, technical debt and an effort estimate); presents the risk register for the project and the product; and sets out the maintenance plan that classifies the changes the system is expected to need after delivery as corrective, adaptive, perfective or preventive.

1.2 References

Boehm, B. W. (1981). *Software engineering economics*. Prentice Hall.

Boehm, B. W. (1991). Software risk management: Principles and practices. *IEEE Software, 8*(1), 32–41. https://doi.org/10.1109/52.62930

Lientz, B. P., & Swanson, E. B. (1980). *Software maintenance management*. Addison-Wesley.

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK guide)* (7th ed.). Project Management Institute.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

2. PROJECT MANAGEMENT

2.1 Process Model

The project followed an incremental, time-boxed process with daily sprints, combined with test-driven development for the security-critical core. Requirements, elicitation and the UML models were produced first, because the design rests on them; implementation then proceeded in increments that each ended with a working, tested build: the core library, the services, the web application, security hardening, then the documents. A plan-driven waterfall was rejected because the deadline was fixed and the risk of finding a design problem late (for example in how the browser and Python agree on signatures) was high; short increments with continuous integration exposed such problems on the day they appeared.

Because this is a solo project, the agile ceremonies were reduced to what adds value for one person: a written sprint plan with a definition of done for each day, a scope cut list agreed in advance, a daily earned-value check against the baseline, and a decision log recording every significant design decision with its reason.

2.2 Work Breakdown and Estimates

The work was broken into packages estimated in story points, where one point is roughly one focused hour (Table 2.1). The total, the budget at completion (BAC), is 68 points.

Table 2.1: Work packages

Package	Content	Points	Status on 4 Nov
WP1	Core cryptographic library, test-first, and CI pipeline	8	Done 30 Oct
WP2	Five back-end services with integration tests	16	Done 3 Nov
WP3	Web application: Verifier, Wallet, Issuer Portal, Audit	12	Done 3 Nov
WP4	Security testing, hardening and generated RTM	6	Done 3 Nov
WP5	Candidate's laptop setup and first push; CI green on GitHub	2	Done 3 Nov
WP6	OWASP ZAP scan, STRIDE table, RTM from test tags; phone test	8	5 of 8: phone test pending
WP7	Documentation: SRS, design, security, test, project plan, user manual, technical report	8	4 of 8: four documents done
WP8	Slides, rehearsal and packaging	4	Planned 7 Nov
Total (BAC)		68	53 earned

2.3 Scope Management

A cut list was agreed at the start, so that falling behind would cut the least valuable features in a known order rather than in a panic. It was used once: when the earned value fell behind (SPI 0.25 on 2 November), in the re-plan of 3 November multi-factor authentication for staff was cut and the Hausa translation was limited to the verification verdicts. Some items were never to be cut: offline verification, challenge–response, revocation, the audit chain, the tests and CI. The cut items are recorded as Partial requirements in the RTM and as perfective maintenance in Section 5.

2.4 Tools

Git and GitHub for version control; GitHub Actions for continuous integration; the decision log, sprint plan and requirements kept as Markdown in the repository and the project workspace; generated documents rebuilt from the repository so that figures, requirements and test results cannot drift apart.

3. DELIVERY METRICS

3.1 Earned Value

Progress was tracked with earned value management (Project Management Institute, 2021). Planned value (PV) is the cumulative number of points the baseline schedule says should be done by a date; earned value (EV) is the cumulative points of work actually completed, credited only when a package's definition of done is met (partly done packages are credited for their completed sub-deliverables). The schedule performance index is SPI = EV / PV, and the schedule variance is SV = EV − PV.

Table 3.1: Earned value by day (story points)

Date	PV	EV	SPI	SV	Note
30 Oct	8	8	1.00	0	Core library and CI on plan
31 Oct	16	8	0.50	-8	No sprint work (other commitments)
1 Nov	24	8	0.33	-16	
2 Nov	32	8	0.25	-24	SPI at its lowest; corrective action
3 Nov	40	36	0.90	-4	Services and web app delivered in one day
4 Nov	56	53	0.95	-3	Security, ZAP scan, four documents
5 Nov	60	–	–	–	Planned
6 Nov	64	–	–	–	Planned
7 Nov	68	–	–	–	Planned

The SPI fell to 0.25 on 2 November, when three planned days passed with no sprint work. The corrective action was to re-plan the remaining days, apply the cut list early, and pull work forward: the web application and the security testing, planned for later days, were delivered on 3 November together with the services. SPI recovered to 0.90 that evening and to 0.95 on 4 November. The remaining variance of three points is the phone test, which is scheduled before the defence.

The cost performance index (CPI = EV / AC) was not computed. It needs the actual cost of the work, and as an unpaid solo project the hours spent were not logged separately from other study; reporting a CPI would mean inventing the denominator.

3.2 Burndown

Figure 3.1 shows the same data as a burndown chart: the points remaining against the plan. The flat section from 31 October to 2 November is the slip; the steep fall on 3 November is the catch-up.

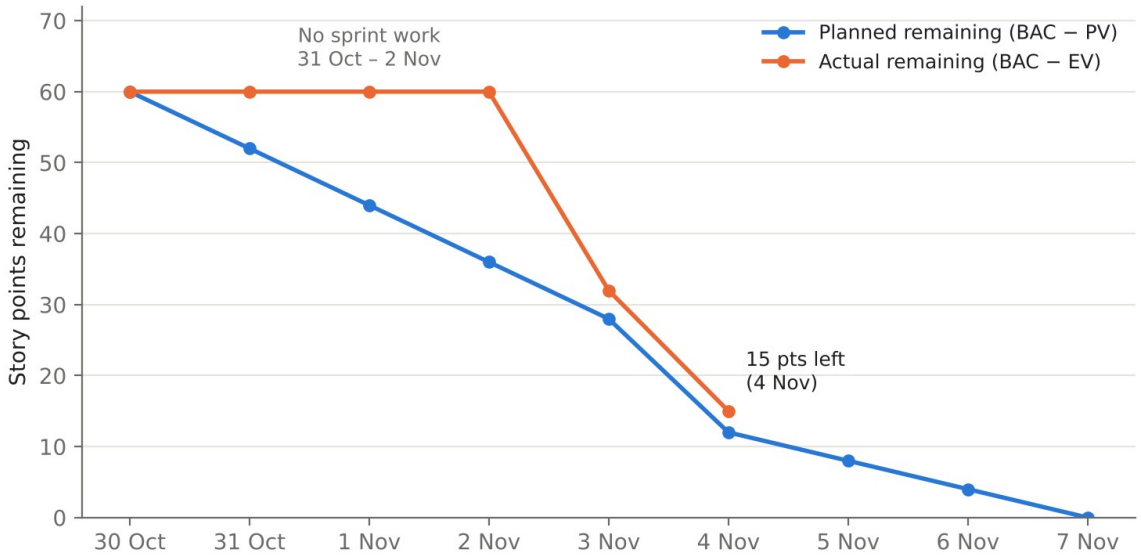


Figure 3.1: Burndown: planned and actual story points remaining

3.3 Velocity

Velocity is the number of points completed per working day. On the three working days so far it was 8 (30 October), 28 (3 November) and 17 (4 November), an average of about 18 points per working day once work resumed. At that rate the 15 remaining points need one more working day, leaving a day of margin before the deadline. The course recommends reserving 15 to 20% of each sprint for refactoring; in practice the refactoring here was done inside the red–green–refactor cycle and in the regenerated documents rather than as a separate allocation.

3.4 Technical Debt Ratio

The technical debt ratio (TDR) compares the cost of fixing known shortcuts with the cost of building the system: TDR = remediation cost / development cost. Development cost is estimated with the convention used by SonarQube, 30 minutes per line of code; the production code has about 2,400 non-blank, non-comment lines, giving about 1,200 hours. The known debt items, with their estimated remediation effort, are in Table 3.2.

Table 3.2: Known technical debt

Item	Remediation (hours)
Upgrade Vite 5 and Vitest 2 to current major versions (development-tool advisories)	4
Move CI actions off Node 20, which GitHub runners are retiring	1
Cache the signed revocation list and re-verify it on every load	3
Store the farmer's signed challenge answer in the audit record	2
Run the Playwright system tests in CI	4
Unit tests for the browser network client, storage helper and router	4
Move the login rate limiter to a shared store for multiple instances	3
Agent PIN unlock for the seven-day offline token	6
Total	27

TDR = 27 / 1,200 ≈ 2.3%. This is within the 5% band that SonarQube rates A. The figure is only as good as the list behind it, which is why each item is also a scheduled maintenance task in Section 5.

3.5 Effort Estimate with COCOMO

For comparison, the basic COCOMO model for an organic project (Boehm, 1981) estimates effort as E = 2.4 × KLOC^1.05 person-months and schedule as T = 2.5 × E^0.38 months. For 2.4 KLOC this gives E ≈ 6.0 person-months and T ≈ 4.9 months. The actual development took a few working days of one person. The gap is expected and worth explaining rather than hiding: COCOMO was calibrated on projects of the 1970s and 1980s, and this project reuses mature libraries for all the cryptography and web framework code, generates its documents from the code, and was developed with an AI assistant (see the AI-use statement in the technical report, QRW-TR-07). COCOMO remains useful as an upper bound for planning a production version, where field trials, integration with NIMC and organisational work would dominate.

3.6 Quality Metrics

Table 3.3: Quality metrics on 4 November 2026

Metric	Value
Automated test cases / checks	94 / 145, all passing
Statement coverage	Core 97%; services 97%; browser verifier 90%
Requirements covered by a passing test or manual verification	48 of 48
Defects found during development / open	7 / 0
Defect density found before delivery	About 2.9 per KLOC (7 / 2.4)
Static analysis and dependency findings open	0
Dynamic scan findings open (High / Medium)	0 / 0

4. RISK MANAGEMENT

4.1 Process

Risks were managed in the four-step cycle taught in the course: identification, analysis, planning and monitoring (Boehm, 1991). Risks were identified from the problem statement, the elicitation findings, the STRIDE threat model and the schedule. Each was analysed by estimating its probability P(risk) between 0 and 1 and its loss L(risk). Following the course, the risk exposure is RE = P(risk) × L(risk). Loss is measured in person-days of effort to recover, so that project and product risks can be compared on one scale; for product risks this is the effort to investigate an incident, re-issue credentials or rework the system. Each risk was then given a mitigation (to reduce P or L) and a contingency, and was reviewed at each daily earned-value check.

The qualitative rating places each risk in a three-by-three matrix of probability (Low below 0.25, Medium 0.25 to 0.5, High above 0.5) and impact (Low below 2 person-days, Medium 2 to 7, High above 7). The product of the two levels gives the tier: Green (1 to 2), Yellow (3 to 4) or Red (6 to 9).

4.2 Risk Register

Table 4.1 lists the eleven risks ranked by exposure; Table 4.2 gives the mitigation and current status of each.

Table 4.1: Risk register ranked by exposure

ID	Type	Risk	P	L (days)	RE	Tier
PR-1	Project	Schedule slip before the 8 November deadline (solo developer; another examination due the same week)	0.5	4	2	Yellow
PD-6	Product	Non-compliance with the Nigeria Data Protection Act 2023	0.2	10	2	Yellow
PD-7	Product	Low adoption by agents and farmers (literacy, shared phones, trust)	0.4	5	2	Yellow
PD-1	Product	Revoked credential accepted at an offline point before the verifier next syncs	0.5	3	1.5	Yellow
PD-2	Product	Issuer signing key stolen, allowing forged identities	0.05	30	1.5	Yellow
PR-2	Project	Live demonstration fails during the oral defence (laptop, network, camera or browser)	0.3	3	0.9	Yellow
PD-4	Product	Verifier too slow or incompatible on low-end Android phones	0.3	3	0.9	Yellow
PD-3	Product	Impostor uses a paper wallet (L1), checked only by face	0.4	2	0.8	Yellow
PR-4	Project	Loss of work through a single failed machine	0.1	5	0.5	Green
PD-5	Product	Vulnerable or outdated dependencies (development tools; Node 20 end of support in CI)	0.5	1	0.5	Green
PR-3	Project	Toolchain differences break the build on the candidate's Windows laptop	0.3	1	0.3	Green

Table 4.2: Risk mitigation and status

ID	Mitigation and contingency	Status
PR-1	Cut list applied early; SPI checked daily; documents generated from the code so they cannot drift; work pulled forward when ahead	Open: SPI 0.95 on 4 Nov
PD-6	Data minimisation by design; NIN hashed; consent step at enrolment; data protection impact assessment before rollout	Open (production)
PD-7	Three-tap Verifier flow; Hausa verdicts; paper wallet for feature-phone users; agent training in the rollout plan	Open (rollout)
PD-1	Verifier warns at 7 days and refuses at 30; credential expiry; sync whenever a signal is available; planned delta lists	Accepted (residual)
PD-2	Key only in the Issuer, encrypted at rest; production HSM; key IDs allow rotation and mass re-issue	Open (production)
PR-2	Whole system runs offline on one laptop; one-click launcher; paste fallback for every QR; Guided Demo; screenshots and the browser test recordings as backup	Open: rehearsal 7 Nov
PD-4	Measured 4.3 ms per check in the browser library; PWA uses standard Web Crypto; phone test before defence	Open: phone test pending
PD-3	Assurance level recorded per check; L2 preferred; agent training; audit anomaly report	Accepted (residual)
PR-4	GitHub remote; dated update bundles; generated documents rebuilt from source	Closed
PD-5	pip-audit and npm audit in CI; dependency removed when advisory found; scheduled adaptive upgrades	Open: monitored
PR-3	Pinned versions; CI on every push; beginner setup guide; fallback install commands	Occurred once (pip on Windows), fixed

4.3 Risk Matrix

Figure 4.1 places the risks on the matrix. No risk is in the Red tier. Most sit in the Medium–Medium cell; two low-probability, high-impact product risks (signing-key theft and data-protection non-compliance) are Yellow because of their impact and would be the first to address before a real rollout.

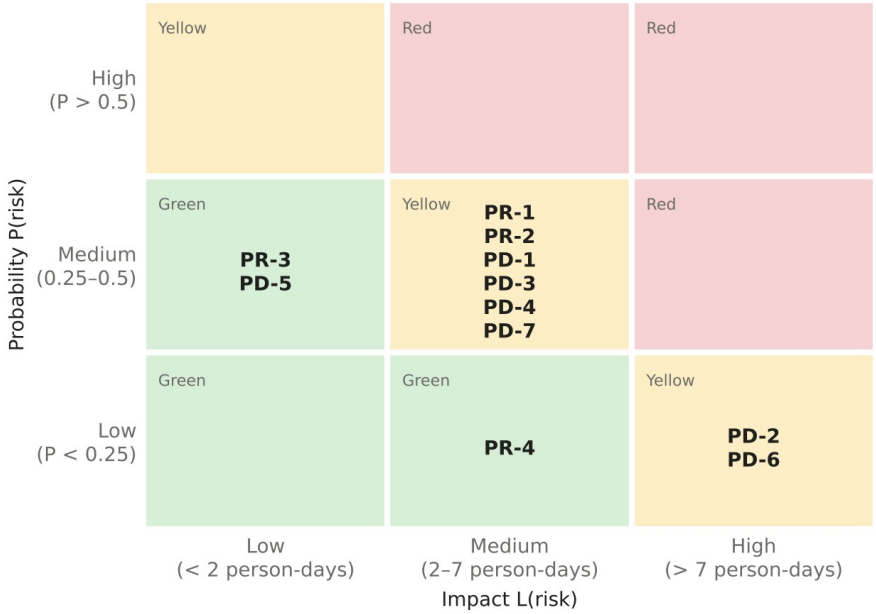


Figure 4.1: Risk matrix (probability × impact)

4.4 Discussion of the Main Risks

Schedule slip (PR-1) shares the highest exposure and is the only risk that already occurred: the SPI fell to 0.25. The mitigations worked as planned (the cut list, pulling work forward, and generating documents from the code) and the SPI is back to 0.95. **Signing-key theft (PD-2)** has a low probability but the largest loss, because every credential would have to be re-issued; the design limits it by keeping the key in one service, and the key identifier in every credential makes rotation possible. **Data-protection non-compliance (PD-6)** is addressed by design through minimisation, but a real rollout also needs a data protection impact assessment and a recorded consent procedure, which are organisational rather than software tasks. **Revocation lag (PD-1)** is the inherent cost of verifying offline; it is bounded by the list-age rules and is accepted as a residual risk, explained in QRW-SEC-03.

5. MAINTENANCE PLAN

5.1 Maintenance Model

After delivery, changes are expected in all four classes defined by Lientz and Swanson (1980) and used in the course. Corrective maintenance fixes faults; adaptive maintenance keeps the system working as its environment changes; perfective maintenance improves it in response to users; and preventive maintenance removes latent faults before they cause failures. Each anticipated change below is classified, prioritised and estimated, so that the plan can be scheduled rather than handled only when something breaks.

5.2 Anticipated Changes

Table 5.1: Maintenance plan by class

Class	Change	Trigger	Priority	Effort
Corrective	Fix any fault reported from field use, starting with a failing test that reproduces it	Bug report or failed check	High	As needed
Adaptive	Move GitHub Actions jobs off Node 20 (deprecation warnings already shown on the runners)	Runner deprecation	High	1 h
Adaptive	Upgrade Vite 5 to the current major and Vitest 2 to the current major; re-run the dependency audit	Development-tool advisories	Medium	4 h
Adaptive	Replace the mock NIMC with the real NIMC interface when access is granted	NIMC agreement	High	3 days
Adaptive	Move from SQLite to PostgreSQL and the issuer key to an HSM for production	Production rollout	High	3 days
Adaptive	Track changes in Android browsers and Nigeria Data Protection Commission guidance	Annual review	Medium	1 day/year
Perfective	Multi-factor authentication for Admin and Auditor accounts	Cut list item	High	1 day
Perfective	Full Hausa translation of every screen	Cut list item; user need	High	4 h + review
Perfective	Agent PIN unlock for the seven-day offline token	Partial FR-14	Medium	6 h
Perfective	Delta revocation lists small enough for SMS or USSD	Reduce revocation lag (PD-1)	Medium	3 days
Perfective	Store the signed challenge answer in the audit record (non-repudiation)	STRIDE R2	Medium	2 h
Perfective	Anomaly report screen and an improved printed paper wallet layout	User feedback	Low	1 day
Preventive	Cache the signed revocation list and re-verify it on load	Residual risk (QRW-SEC-03)	High	3 h
Preventive	Run the Playwright system tests and the ZAP scan in CI	Testing limitation	Medium	1 day
Preventive	Unit tests for the browser network client, storage helper and router	Coverage gap	Medium	4 h
Preventive	Automated dependency update pull requests (for example Dependabot)	Supply-chain hygiene	Medium	1 h
Preventive	Key-rotation drill and mutual TLS between services	PD-2; multi-host deployment	Medium	2 days

5.3 Maintenance Process

Every change, whatever its class, follows the same steps, reusing the engineering infrastructure built for development:

1. **Request.** Record the change as a GitHub issue with its class and priority.
2. **Impact analysis.** Use the RTM to find the requirements, design elements and tests the change touches.
3. **Change test-first.** Write or update the failing test, then the code, then refactor.
4. **Regression.** CI runs all tests, coverage gates, static analysis and dependency scans; the change cannot merge if any fails.
5. **Release and record.** Regenerate the RTM and documents, tag the release, and update the decision log if the design changed.

5.4 Schedule and Effort Allocation

In the first month after submission: the adaptive CI and tooling upgrades and the high-priority preventive item (re-verifying the cached list). Before any pilot: multi-factor authentication, full Hausa, the real NIMC interface, PostgreSQL and an HSM. During a pilot: delta revocation lists and improvements from field feedback. Following the course guidance, 15 to 20% of each maintenance cycle is reserved for preventive work and refactoring, so that the technical debt ratio stays below 5%.

6. SUMMARY

The project was run as short, test-driven increments with a written plan, a cut list and daily earned-value tracking. When the schedule slipped (SPI 0.25), the plan was corrected and the SPI recovered to 0.95, with one outstanding item, the phone test. Eleven project and product risks are registered with their exposure, mitigation and status; none is in the Red tier. The maintenance plan anticipates seventeen changes across all four classes, each with a trigger, priority and effort, and a single process that keeps every change tested and traceable.